CloneScope

Git Basics & Practical Progress Log

Student: Raian Huq (BSSE1708) Program: SPL-1, IIT, University of Dhaka

Learning session: 25 October 2026 Working branch: chore/repo-foundation

Purpose

Record the Git concepts I learned, the commands I actually ran, and the genuine progress completed for my CloneScope project. This document is a learning artifact; it is not evidence that Type-1 or Type-2 detection has been implemented.

What I actually completed

• Opened the existing local SPL-1 project directory in PowerShell and VS Code.

• Checked git status; confirmed the main branch was clean and up to date with the last-known origin/main.

• Verified the remote URL using git remote -v and refreshed remote information using git fetch origin.

• Created and published a new branch named chore/repo-foundation.

• Improved README.md to explain CloneScope and the approved midterm scope.

• Added .gitignore to exclude generated executable, object, CMake build, and temporary files.

• Created src/main.cpp, compiled it using C++17, and executed the CLI welcome screen.

• Staged, committed, and synchronized each of these three changes through VS Code.

Verified executable output

```
CloneScope v0.1
Code Clone Analysis Tool
Status: Development Started
```

Important: the feature branch currently contains the work. A pull request or merge into main has not yet been documented.

Git concepts: what the words mean

Git keeps a local history of project changes. GitHub hosts a remote copy of that repository. A branch lets me develop without directly editing the stable main branch.

Concept	Meaning in this project
Working tree	Files currently being edited in my local CloneScope folder.
Untracked (U)	A new file, such as src/main.cpp before it was added to Git.
Modified (M)	An existing tracked file, such as README.md after I changed it.
Stage / git add	Select changed files that belong in the next snapshot.
Commit	Record a meaningful local snapshot with an explanatory message.
Push	Upload local commits to the tracked GitHub branch.
Fetch	Download remote branch/commit information without changing the working files.
Origin	The short name for the GitHub remote URL.
Branch	An independent development line; mine is chore/repo-foundation.
Sync Changes	VS Code action that pulls/fetches as needed and pushes local commits.

Commands I used or observed

`git status`

Showed branch main, clean working tree, and tracked remote status.

`git remote -v`

Showed origin pointing to github.com/raian-x8/SPL_01_BSSE1708.git for fetch/push.

`git fetch origin`

Checked for remote changes before creating the new branch.

`git switch -c chore/repo-foundation`

Created a separate local branch and switched to it.

`git push -u origin chore/repo-foundation`

Published the branch and configured upstream tracking.

`g++ -std=c++17 src/main.cpp -o clonescope.exe`

Compiled the C++17 starter program.

`.\clonescope.exe`

Ran the compiled program on Windows PowerShell.

My practical workflow: step by step

Order	Action	Evidence / result
01	Open local folder	C:\Users\raian\Developer\SPL_01_BSSE1708
02	Check current Git state	Branch main; working tree clean.
03	Verify remote	origin URL matches my GitHub SPL-1 repository.
04	Fetch current remote state	main remained up to date with origin/main.
05	Create and push feature branch	chore/repo-foundation created and tracked on GitHub.
06	Write project README	Commit: docs: introduce CloneScope project in README
07	Configure ignored output	Commit: chore: add Git ignore rules for C++ builds
08	Create starter C++ application	Commit: feat: initialize CloneScope C++17 CLI application
09	Compile and run	Welcome message printed successfully; no compilation errors.
10	Synchronize with GitHub	Outgoing-commit indicators cleared in VS Code.

Why my commits are meaningful

Each commit corresponds to a distinct, reviewable result: project description, repository configuration, or executable source code. These are real changes, rather than empty or artificial commits.

How I should continue working

- Learn the relevant concept, then implement a small change.
- Build or test before recording a code change as complete.
- Stage only files relevant to that change; choose a clear feat:, fix:, test:, docs:, or chore: message.
- Commit after each meaningful milestone and push during the development session.
- Keep dated learning notes, reproducible test cases, known limitations, and references.
- Never backdate commits, create empty filler commits, or claim unfinished features are complete.

Next learning milestones for CloneScope

This section is a plan, not work already completed. The supervisor-approved midterm target prioritizes a reliable Type-1/Type-2 core with initial stylometry, basic statistics, and export.

Stage	Concepts to understand	Next build evidence
File input	std::ifstream, std::filesystem, errors	Read small local C files and report filename/line count.
Preprocessing	Comments, strings, lexical states, line mapping	Handle tricky comments while preserving locations.
Tokens	Keywords, identifiers, literals, operators	Print a token stream from controlled inputs.
Type-1	Whitespace/comment normalization, hashing, collision checking	Find an exact normalized match with original line ranges.
Type-2	Canonical identifiers, one-to-one consistent renaming	Detect renamed functions and reject inconsistent renaming.
Baseline output	Clone tags, initial style features, CSV/JSON	Export auditable basic matches and student-pair summaries.
Later: Type-3	Token n-grams, Jaccard, LCS, edit distance	Post-midterm edited near-miss analysis.

Reflection: what I learned

Git and GitHub are not the same: Git tracks project history locally; GitHub hosts the remote repository. Staging selects a change, committing records it, and pushing uploads it. My feature branch allows safe work while main remains a stable reference. I can also record authentic learning as documentation when it directly supports development.

References and evidence

Git official book: https://git-scm.com/book/en/v2

Git reference manual: https://git-scm.com/docs

GitHub Git guide: https://docs.github.com/en/get-started/using-git

My repository: https://github.com/raian-x8/SPL_01_BSSE1708

Working branch: https://github.com/raian-x8/SPL_01_BSSE1708/tree/chore/repo-foundation

Suggested PDF commit (after it is added to the repository)

```
docs: add Git basics learning and setup progress log
```

Place this PDF in docs/learning-notes/ to keep learning artifacts separate from executable code.